



Home

Laborator: Compilatoare

- Back to [PLP](#)

Exerciții de antrenament:

- Studiați cu atenție compilatorul pentru expresii discutate la curs. Rulați din nou exemplele și demonstrațiile discutate.

Exerciții propuse:

În acest laborator vom lucra cu un limbaj B pentru expresii booleene care va conține constantele T și F (corespunzătoare valorilor *true* și respectiv *false*) și operațiile: negație, conjuncție și disjuncție. Sintaxa abstractă este definită mai jos:

```
Require Import String.
Require Import List.
Import ListNotations.
```

```
Inductive B :=
| T : B
| F : B
| neg : B -> B
| and : B -> B -> B
| or  : B -> B -> B.
```

1. (1 punct) Definiți un interpretor (o funcție `interpret` cu semnatura `interpret (b : B) : bool`) pentru limbajul B .
2. (1 punct) Se consideră definiția unui *stack machine* de mai jos:

```
Inductive Instruction :=
| push : nat -> Instruction
| inv  : Instruction
| add  : Instruction
| mul  : Instruction.
```

Deși această mașină de calcul bazat pe stivă lucrează cu numere naturale, pe stivă vom utiliza **doar** valorile 0 și 1 (sau 0 și s 0). Motivul pentru care ne limităm la aceste valori este că vom vrea să definim un compilator pentru expresiile limbajului B în care valorile T și F să fie codificate cu 1 și respectiv 0. Mai mult, operațiile booleene conjuncție și disjuncție vor fi codificate utilizând operații peste numere naturale (`mul` și respectiv `add`), iar negația va fi codificată utilizând operația `inv` (care transformă 1 în 0 și invers). Așadar, definim următoarea semantică pentru instrucțiunea `push`:

```

Fixpoint run_instruction (i : Instruction) (stack : list nat) : list nat :=
  match i with
  | push n => match n with
    | 0 => 0 :: stack
    | S n' => (S 0) :: stack
    end
  | inv => ...
  | add => ...
  | mul => ...
  end.

```

Cerință: completați spațiile libere (...) din definiția de mai sus.

3. (1 punct) Completați funcția de mai jos care primește la intrare o listă instrucțiuni și o stivă și returnează stiva obținută în urma execuției instrucțiunilor:

```

Fixpoint run_instructions
  (stpgm : list Instruction)
  (stack : list nat) :=
  ...

```

Această funcție va apela funcția `run_instruction` de la exercițiul precedent.

4. (1 punct) Completați funcția de compilare de mai jos:

```

Fixpoint compile (b : B) : list Instruction :=
  match b with
  ...
  end.

```

5. (2 puncte) Demonstrați următoarele proprietăți:

```

Lemma soundness_helper:
  forall b instrs stack,
    run_instructions ((compile b) ++ instrs) stack =
    run_instructions instrs (to_nat (interpret b) :: stack).

```

Proof.
Admitted.

```

Theorem soundness :
  forall b,
    run_instructions (compile b) nil = [to_nat (interpret b)].

```

Proof.
Admitted.

Pentru a înțelege mai bine cum puteți aborda acest exercițiu, citiți cu atenție demonstrațiile făcute la curs.

Coq Reference Manual: <https://coq.inria.fr/distrib/current/refman/>

Last updated: 01/09/2025 09:05:10 - Copyright © Andrei Arusoae

Facultatea de Informatică, Universitatea „Alexandru Ioan Cuza” din Iași